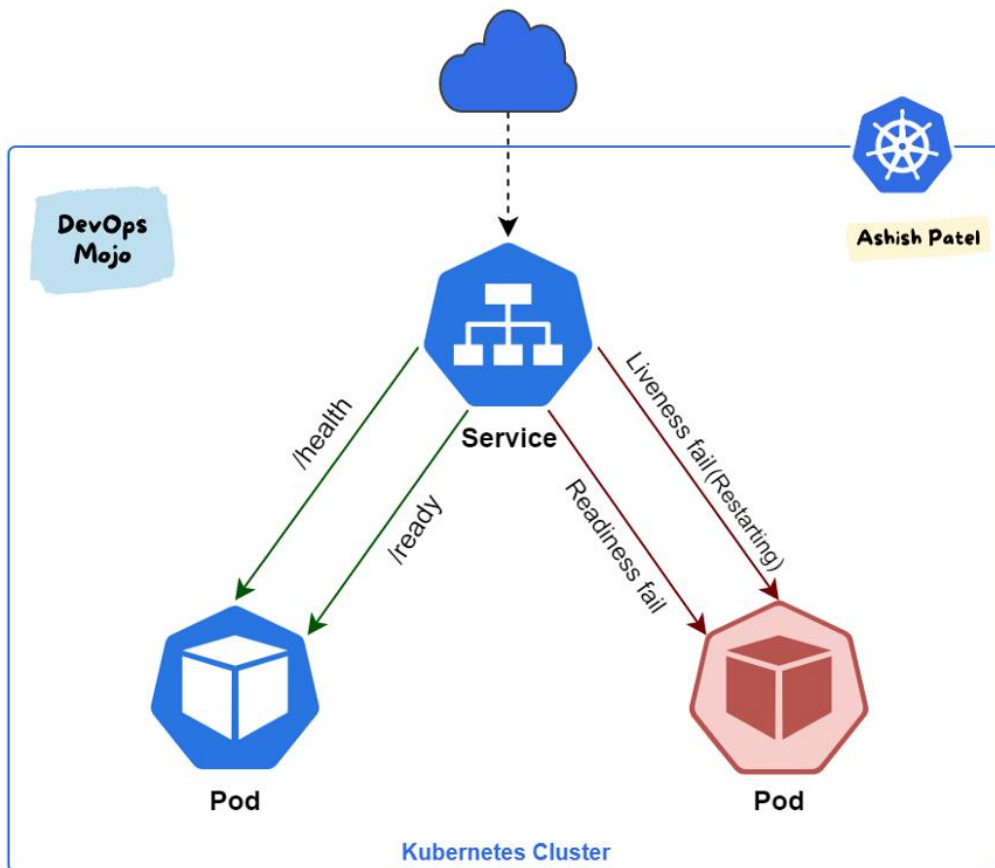


Kubernetes — Probes (Liveness, Readiness, and Startup) Overview

Introduction to Types of Probes and Configure Health Checks using Probes in Kubernetes.



Kubernetes provides probes (health checks) to monitor and act on the state of Pods (Containers) and to make sure only healthy Pods serve traffic. With help of Probes, we can control when a pod should be deemed started, ready for service, or live to serve traffic.

Probes are used to detect:

- Containers that haven't started yet and can't serve traffic.
- Containers that are overwhelmed and can't serve additional traffic.
- Containers that are completely dead and not serving any traffic.

Types of Probes

Kubernetes gives you three types of health checks probes:

1. Liveness
2. Readiness

Liveness Probe

- Liveness probes let Kubernetes know whether your app (running in a container inside Pod) is healthy.
- Indicates whether the container is running.
- If app is healthy, Kubernetes will not interfere with pod functioning. If app is unhealthy, Pod will be marked as unhealthy. If a Pod fails health-checks continuously, the Kubernetes terminates the pod and starts a new one.
- Liveness probes could catch a deadlock, where an application is running, but unable to make progress. Restarting a container in such a state can help to make the application more available despite bugs.

- Kubelet uses liveness probes to know when to restart a container. If the liveness probe fails, the kubelet kills the container, and the container is subjected to its [restart policy](#).
- Configure a health check that periodically checks if your application is still alive. Containers can be restarted if the check fails.
- Kubelet will periodically send a GET request to the Pod and expect a response; if the Pod responds between 200–399 then the kubelet considers that Pod is healthy; otherwise, the Pod will be marked as unhealthy.

Use Cases

- Use Liveness Probe to remove unhealthy pods.
- If you'd like your container to be killed and restarted if a probe fails, then specify a liveness probe, and specify a `restartPolicy` of Always or OnFailure.
- Use a liveness probe for a container that can fail with no possible recovery.
- If your container cannot crash by itself when there is an unexpected error occur, then use liveness probes. Using liveness probes can overcome some of the bugs the process might have.

Recommendation

The recommended way to implement the liveness probe is for application to expose a `/health` HTTP endpoint. When receiving a request on this endpoint, the application should send a `200`

OK" response if it is considered healthy. Healthy means the following:

- Application is running.
- Its main dependencies are met (e.g. app can access database).

Readiness Probe

- Readiness probes let Kubernetes know when your app (running in a container inside Pod) is ready to serve traffic.
- Indicates whether the container is ready to respond to requests.
- Kubernetes makes sure the readiness probe passes before allowing a service to send traffic to the pod.
- Unlike a liveness probe, a readiness probe doesn't kill the container. If the readiness probe fails, Kubernetes simply hides the container's Pod from corresponding Services, so that no traffic is redirected to it.
- Liveness probes do not wait for readiness probes to succeed. If you want to wait before executing a liveness probe you should use `initialDelaySeconds` or `startupProbe`.
- A side-effect of using Readiness Probes is that they can increase the time it takes to update Deployments.

Use Cases

- Use Readiness Probe to detect partial unavailability.
- If you'd like to start sending traffic to a Pod only when a probe succeeds, specify a readiness probe.

- If you want your container to be able to take itself down for maintenance, you can specify a readiness probe that checks an endpoint specific to readiness that is different from the liveness probe.

Recommendation

The recommended way to implement the readiness probe is for your application to expose a `/ready` HTTP endpoint. When it receives a request on this endpoint, the application should send a `"200 OK"` response if it is ready to receive traffic. Ready to receive traffic means the following:

- Application is healthy.
- Any potential initialization steps are completed.
- Any valid request sent to the application doesn't result in an error.

Feature	Liveness Probe	Readiness Probe
Purpose	Checks if the app is <i>still alive</i>	Checks if the app is <i>ready to serve traffic</i>
Failure Action	Pod/container is restarted by Kubelet	Pod is removed from Service endpoints , no traffic sent
When it runs	Continuously during pod's lifecycle (after startupProbe, if defined)	Continuously during pod's lifecycle
Effect on Traffic	No impact directly — only restarts pod	Stops sending traffic to pod until it's ready
Use Case	App crash, deadlock, memory leak, etc.	App loading data, waiting for DB, etc.

Liveness Probe — Use when:

Scenario	Example
App hangs or crashes silently	Java app stuck in an infinite loop
App enters a deadlock state	Thread pool exhausted
App keeps responding 500 or times out	Flask app <code>/health</code> returns 500
Memory leak or internal failure causes it to stop working properly	REST API failing due to internal corruption

Readiness Probe — Use when:

Scenario	Example
App needs time to initialize	Loading ML model or DB cache
External service/database is temporarily unavailable	Waiting for PostgreSQL to come online
Rolling deployment: new pods aren't ready instantly	Prevents service disruption
App is healthy but not yet ready to serve	Readiness returns 503, app not in endpoints